

Fundamentos de ciencia de datos con R - Módulo 7

Clase 3: k-Nearest Neighbors

CEPAL - Unidad de Estadísticas Sociales

2025-11-20

Introducción

El método de los **k vecinos más cercanos (k-Nearest Neighbors, k-NN)** es un algoritmo de aprendizaje supervisado muy sencillo e intuitivo.

La idea básica es:

*Para predecir el valor (o clase) de una observación nueva, miramos a sus **k vecinos más cercanos** en los datos y usamos lo que ocurre en ese vecindario.*

- ▶ Si la respuesta es **categorica** → votación de mayoría (clasificación).
- ▶ Si la respuesta es **numérica** → promedio de los vecinos (regresión).

Intuición del método k-NN



Idea principal

“Dime con quién andas y te diré quién eres”:

k-NN clasifica una observación según las características de sus vecinos más cercanos.

Intuición del método k-NN

Pasos generales:

1. Tenemos un conjunto de datos con variables explicativas y una respuesta.
2. Para una nueva observación:
 - ▶ Calculamos su **distancia** a todas las observaciones de entrenamiento.
 - ▶ Identificamos los **k vecinos más cercanos**.
3. Clasificación:
 - ▶ Asignamos la clase más frecuente entre los k vecinos.
4. Regresión:
 - ▶ Tomamos el promedio de la respuesta de los k vecinos.

Distancia y normalización

En k-NN, la noción de “cercanía” depende de cómo definimos la **distancia**.

La más usada es la **distancia Euclídea**:

$$d(i, j) = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \dots}$$

Problema:

- ▶ Si una variable está en una escala muy grande (por ejemplo, gasto en pesos) y otra en escala pequeña (edad en años), la variable grande domina la distancia.

Solución:

- ▶ **Estandarizar / normalizar** las variables numéricas antes de aplicar k-NN (por ejemplo, usando `scale()` en R).

Elección del parámetro k

El parámetro k controla la complejidad del modelo:

- ▶ k muy pequeño (ej. 1 ó 3):
 - ▶ El modelo es muy flexible, pero puede **sobreajustarse** (sensible al ruido).
- ▶ k muy grande (ej. 50, 100):
 - ▶ El modelo es más estable, pero puede **subajustarse** (pierde detalles).

En la práctica:

- ▶ Se prueban varios valores de k y se elige el que da mejor desempeño (ej. mejor exactitud en validación cruzada).

Ejemplo práctico: clasificación con k-NN

Usaremos la base `base_personas_gasto.rds` para predecir si un hogar tiene **ingreso bajo** (por debajo de la mediana).

```
library(tidyverse)
library(class)
datos <- readRDS("../Data/base_personas_gasto.rds")
set.seed(123)

muestra <- datos %>%
  slice_sample(n = 400) %>%
  mutate(
    ingreso_bajo = if_else(
      ingreso < median(ingreso, na.rm = TRUE),
      "Sí", "No"
    ),
    ingreso_bajo = factor(ingreso_bajo)
  ) %>%
  select(ingreso_bajo, edad, sexo, gasto_hh, area, pobreza, ingreso)
```

Ejemplo práctico: clasificación con k-NN

Usaremos la base `base_personas_gasto.rds` para predecir si un hogar tiene **ingreso bajo** (por debajo de la mediana).

```
head(muestra, 10)
```

ingreso_bajo	edad	sexo	gasto_hh	area	pobreza	ingreso
No	35	Hombre	7236.040	1	3	2412.013
No	65	Mujer	12036.117	2	3	6018.058
Sí	58	Hombre	5414.580	1	3	1804.860
No	27	Hombre	7923.053	1	3	3961.527
No	38	Hombre	9714.580	2	3	3238.193
No	80	Mujer	5728.647	2	3	2864.323
No	58	Hombre	13989.580	1	3	4663.193
Sí	60	Hombre	11139.010	2	3	1856.502
Sí	17	Mujer	13910.910	1	3	1738.864
Sí	19	Hombre	6281.937	1	3	1256.387

Separación en entrenamiento y prueba

Una vez que tenemos las covariables en formato numérico, dividimos la información en un conjunto de **entrenamiento** (para ajustar el modelo) y otro de **prueba** (para evaluar su desempeño fuera de la muestra).

Separación en entrenamiento y prueba

```
set.seed(123)
n <- nrow(muestra) # N° observaciones
id_train <- sample(seq_len(n), size = 0.7 * n)

train <- muestra[id_train, ]
test  <- muestra[-id_train, ]
table(train$ingreso_bajo)
```

No	Sí
132	148

```
table(test$ingreso_bajo)
```

No	Sí
68	52

Estandarización de las variables

Este método requiere que **todas las covariables sean numéricas**, porque k-NN se basa en calcular distancias entre observaciones.

Por esta razón, las variables categóricas como sexo y area deben ser transformadas a variables numéricas (por ejemplo, mediante dummies) antes de aplicar el algoritmo.

A continuación realizamos la transformación y la estandarización de las covariables.

Estandarización de las variables

```
# Matrices numéricas (model.matrix crea dummies para las categóricas)
x_train <- model.matrix(~ sexo + gasto_hh + area, data = train)[, -1]
x_test  <- model.matrix(~ sexo + gasto_hh + area, data = test)[, -1]

# Variable respuesta
y_train <- train$ingreso_bajo
y_test  <- test$ingreso_bajo
```



Nota

Las funciones `scale()` y `knn()` solo aceptan matrices numéricas.

Por eso primero usamos `model.matrix()` para transformar las variables categóricas (`sexo`, `area`) en dummies y luego estandarizamos.

Estandarización de las variables

```
x_train_scaled <- scale(x_train)
x_test_scaled  <- scale(
  x_test,
  center = attr(x_train_scaled, "scaled:center"),
  scale  = attr(x_train_scaled, "scaled:scale")
)
```

Ajustar k-NN y evaluar

```
set.seed(123)
pred_knn5 <- knn(
  train = x_train_scaled,
  test  = x_test_scaled,
  cl     = y_train,
  k      = 5
)
```

Ajustar k-NN y evaluar

```
conf_matrix <- table(Observado = y_test, Predicho = pred_knn5)
conf_matrix
```

Observado/Predicho	No	Sí
No	47	21
Sí	13	39

Interpretación del resultado k-NN

Matriz de confusión (conf_matrix)

Resume cuántos casos fueron clasificados correctamente o incorrectamente: - **47**

verdaderos negativos (No $\rightarrow$ No)

- **39** verdaderos positivos (Sí $\rightarrow$ Sí)

- **21** falsos positivos (No $\rightarrow$ Sí)

- **13** falsos negativos (Sí $\rightarrow$ No)

Ajustar k-NN y evaluar

```
exatitud <- mean(pred_knn5 == y_test)
exatitud
```

```
[1] 0.7166667
```



Interpretación del resultado k-NN

Exactitud (`mean(pred_knn5 == y_test)`)

Mide la proporción total de aciertos del modelo.

Exactitud = 0.72 → El modelo acierta el **72 %** de las predicciones.

Ventajas y desventajas de k-NN

Ventajas	Desventajas
Método simple e intuitivo	Costoso en cómputo con bases grandes
No requiere suponer una forma funcional	Sensible a la escala de las variables
Se adapta bien a relaciones no lineales	Difícil de interpretar (no hay “modelo” explícito)
Fácil de implementar y explicar	Afectado por variables irrelevantes y ruido

Conclusiones de la clase

Tema	Idea principal	Herramienta
k vecinos más cercanos	Clasifica usando la información de los vecinos más cercanos	<code>class::knn()</code>
Distancia	Define qué tan “cerca” están dos observaciones	Distancia Euclídea
Normalización	Evita que una variable domine la distancia	<code>scale()</code>
Elección de k	Controla complejidad: k pequeño sobreajusta, k grande suaviza	Búsqueda por prueba y error
Evaluación del modelo	Se mide con exactitud y matriz de confusión	<code>table()</code> , <code>mean()</code>